

Appendix A. Installation Manual

A.1 Prerequisites

- AWS account and an authorized IAM identity.
- AWS CLI version 2, Terraform 1.6.0 or later, Git, editor, and browser.
- Project files including main.tf, variables.tf, outputs.tf, .terraform.lock.hcl, and site/index.html.

A.2 AWS authentication

```
aws configure --profile iu-cloud
$env:AWS_PROFILE = "iu-cloud"
aws sts get-caller-identity
```

Use an approved profile, IAM Identity Center session, or environment-based credential mechanism. Confirm the expected account before Terraform operations. Do not save credentials in the repository.

A.3 Initialize, format, validate, plan, and apply

```
cd "D:\Documents\IU\Projects\CloudDep\iu-cloud-programming"
terraform init
terraform fmt -recursive
terraform fmt -check -recursive
terraform validate
terraform plan -out=tfplan
terraform show -no-color tfplan > evidence\terraform-plan.txt
terraform apply tfplan
```

Review the plan before applying. Expected changes are limited to S3, CloudFront, OAC, the bucket policy, and the HTML object. Do not apply a plan containing compute instances, databases, IAM users, access keys, Route 53, or load balancers.

Appendix A. Installation Manual (continued)

A.4 Retrieve and test the URL

```
terraform output -raw website_url
curl.exe -I $(terraform output -raw website_url)
curl.exe -I http://$(terraform output -raw cloudfront_domain_name)
aws s3api get-public-access-block --bucket $(terraform output -raw s3_bucket_name)
curl.exe -I https://$(terraform output -raw s3_origin_domain)/index.html
aws cloudfront get-distribution --id $(terraform output -raw cloudfront_distribution_id) --query
"Distribution.Status"
```

Expected results are HTTPS 200 through CloudFront, HTTP redirect to HTTPS, four true Public Access Block values, HTTP 403 from direct S3, and Deployed distribution status.

A.5 Update the page and invalidate CloudFront

```
# Edit site/index.html
terraform plan -out=tfplan
terraform apply tfplan
aws cloudfront create-invalidation --distribution-id $(terraform output -raw cloud-
front_distribution_id) --paths "/*"
terraform plan
```

Wait until the invalidation status is Completed, refresh the website, verify the visible change, and confirm the final plan contains no unexpected differences.

A.6 Destroy the temporary environment

```
terraform destroy
```

Review the destruction plan carefully. Confirm the CloudFront distribution, OAC, bucket policy, object, and bucket are removed. `force_destroy` is acceptable only for this temporary assessment environment.

Appendix A. Troubleshooting and Security Warnings

A.7 Troubleshooting

- Path not found when redirecting plan output: change to `iu-cloud-programming` and confirm `evidence/` exists.
- No visible output from terraform show redirection: successful redirection writes to the file; open `evidence/terraform-plan.txt`.
- CloudFront still serves old content: create an invalidation, wait for Completed, and refresh without relying on a stale browser cache.
- AccessDenied during Terraform: confirm the AWS profile, account identity, region, and required permissions.
- Bucket name conflict: retain the account-ID naming logic and verify the authenticated account.
- State lock or interrupted operation: inspect the running process and state backend before taking any recovery action; never delete state blindly.

A.8 Security warnings

- Never commit AWS access keys, secret keys, session tokens, Terraform state, plan files, `.env` files, PEM files, or private keys.
- Do not make the S3 bucket public to solve an origin-access problem. Diagnose OAC, bucket policy, and SourceArn instead.
- Do not share screenshots that contain credentials or session tokens.
- Do not run terraform destroy against the wrong workspace, account, or production environment.
- Do not use `force_destroy` for important production data.
- Re-run the credential-pattern scan before every snapshot and final package.